

Dynamic Programming - Graph Algorithms

Bellman-Ford, Floyd-Warshall, Warshall's Algorithm

Exercise 1: Bellman-Ford Algorithm

Aim:

To write a Python program that finds the shortest distance from a source vertex to all other vertices in a weighted directed graph using the Bellman-Ford algorithm, and reports a negative weight cycle if one exists.

Algorithm:

1. Initialize the distance of the source vertex as 0 and the distance of all other vertices as infinity.
2. Relax all edges (u, v, w): if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v] = \text{dist}[u] + w$. Repeat this relaxation for all edges (V - 1) times, where V is the number of vertices.
3. After (V - 1) iterations, perform one additional pass over all edges to check whether any distance can still be reduced.
4. If any edge can still be relaxed in this extra pass, the graph contains a negative weight cycle.
5. Otherwise, the $\text{dist}[]$ array holds the shortest distance from the source to every other vertex.

Python Code:

```
def bellman_ford(vertices, edges, source):
    dist = [float('inf')] * vertices
    dist[source] = 0
    for _ in range(vertices - 1):
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            return None # Negative weight cycle detected
    return dist

# Example 1
vertices = 5
edges = [(0,1,-1),(0,2,4),(1,2,3),(1,3,2),(1,4,2),(3,2,5),(3,1,1),(4,3,-3)]
result = bellman_ford(vertices, edges, 0)
if result is None:
    print("Negative Weight Cycle Detected")
else:
    for i, d in enumerate(result):
        print(f"Vertex {i}\tDistance {d}")

# Example 2
vertices2 = 3
edges2 = [(0,1,1),(1,2,-1),(2,0,-1)]
result2 = bellman_ford(vertices2, edges2, 0)
print("Negative Weight Cycle Detected" if result2 is None else result2)
```

Input:

Example 1:
Vertices = 5

```
Edges = [(0,1,-1),(0,2,4),(1,2,3),(1,3,2),(1,4,2),(3,2,5),(3,1,1),(4,3,-3)]
Source = 0
```

Example 2:

```
Vertices = 3
```

```
Edges = [(0,1,1),(1,2,-1),(2,0,-1)]
```

```
Source = 0
```

Output:

Example 1:

```
Vertex 0 Distance 0
```

```
Vertex 1 Distance -1
```

```
Vertex 2 Distance 2
```

```
Vertex 3 Distance -2
```

```
Vertex 4 Distance 1
```

Example 2:

```
Negative Weight Cycle Detected
```

Result:

The Python program was executed successfully. For Example 1, the shortest distances from source vertex 0 were correctly computed. For Example 2, the program correctly detected the presence of a negative weight cycle.

Exercise 2: Floyd-Warshall Algorithm

Aim:

To write a Python program that finds the shortest distances between every pair of vertices in a weighted graph represented as an adjacency matrix using the Floyd-Warshall algorithm.

Algorithm:

1. Initialize the distance matrix $\text{dist}[][]$ as the given adjacency matrix, where $\text{dist}[i][j]$ is the direct edge weight (or infinity if no edge exists, and 0 when $i = j$).
2. For every vertex k from 0 to $V - 1$, treat k as a possible intermediate vertex.
3. For every pair of vertices (i, j) , update $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.
4. Repeat step 3 for every pair (i, j) after fixing each intermediate vertex k .
5. After all vertices have been considered as intermediates, $\text{dist}[][]$ contains the shortest distance between every pair of vertices.

Python Code:

```
INF = float('inf')

def floyd_warshall(graph):
    n = len(graph)
    dist = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist

# Example 1
graph = [
    [0, 5, INF, 10],
    [INF, 0, 3, INF],
    [INF, INF, 0, 1],
    [INF, INF, INF, 0]
]
for row in floyd_warshall(graph):
    print(row)

# Example 2
graph2 = [
    [0, 2, INF],
    [INF, 0, 4],
    [1, INF, 0]
]
for row in floyd_warshall(graph2):
    print(row)
```

Input:

Example 1:
[[0,5,INF,10],
[INF,0,3,INF],

```
[INF,INF,0,1],  
[INF,INF,INF,0]]
```

Example 2:

```
[[0,2,INF],  
 [INF,0,4],  
 [1,INF,0]]
```

Output:

Example 1:

```
[0, 5, 8, 9]  
[INF, 0, 3, 4]  
[INF, INF, 0, 1]  
[INF, INF, INF, 0]
```

Example 2:

```
[0, 2, 6]  
[5, 0, 4]  
[1, 3, 0]
```

Result:

The Python program was executed successfully and the all-pairs shortest distance matrices for both examples were correctly computed using the Floyd-Warshall algorithm.

Exercise 3: Warshall's Algorithm

Aim:

To write a Python program that computes the transitive closure of a directed graph represented as an adjacency matrix, to determine whether a path exists between every pair of vertices, using Warshall's algorithm.

Algorithm:

1. Initialize the reachability matrix `reach[][]` as a copy of the given adjacency matrix.
2. For every vertex `k` from 0 to `V - 1`, treat `k` as a possible intermediate vertex.
3. For every pair of vertices `(i, j)`, update `reach[i][j] = reach[i][j] OR (reach[i][k] AND reach[k][j])`.
4. Repeat step 3 for every pair `(i, j)` after fixing each intermediate vertex `k`.
5. The resulting matrix `reach[][]` indicates whether a path exists between every pair of vertices (1 if reachable, 0 otherwise).

Python Code:

```
def warshall(graph):
    n = len(graph)
    reach = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    return reach

# Example 1
graph = [
    [0,1,0,0],
    [0,0,1,0],
    [0,0,0,1],
    [0,0,0,0]
]
for row in warshall(graph):
    print([int(x) for x in row])

# Example 2
graph2 = [
    [0,1,1],
    [0,0,1],
    [1,0,0]
]
for row in warshall(graph2):
    print([int(x) for x in row])
```

Input:

Example 1:
[[0,1,0,0],
[0,0,1,0],
[0,0,0,1],
[0,0,0,0]]

Example 2:

```
[[0,1,1],  
 [0,0,1],  
 [1,0,0]]
```

Output:

Example 1:

```
[0, 1, 1, 1]  
[0, 0, 1, 1]  
[0, 0, 0, 1]  
[0, 0, 0, 0]
```

Example 2:

```
[1, 1, 1]  
[1, 1, 1]  
[1, 1, 1]
```

Result:

The Python program was executed successfully and the transitive closure matrix showing reachability between every pair of vertices was correctly computed for both examples.

Exercise 4: Find the City with the Smallest Number of Reachable Cities

Aim:

To write a Python program that finds the city which can reach the smallest number of other cities within a given distance threshold, using the Floyd-Warshall algorithm to compute all-pairs shortest paths.

Algorithm:

1. Build a $V \times V$ adjacency matrix from the given weighted edges, initializing the distance of a vertex to itself as 0 and all other distances as infinity.
2. For every vertex k from 0 to $V - 1$, treat k as a possible intermediate vertex.
3. For every pair of vertices (i, j) , update $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.
4. Repeat step 3 for every pair (i, j) after fixing each intermediate vertex k .
5. After all vertices have been considered as intermediates, $\text{dist}[][]$ contains the shortest distance between every pair of vertices.
6. For every city, count the number of other cities reachable within the given distance threshold using the computed shortest-path matrix.
7. Return the city with the smallest such count; if there is a tie, return the city with the greatest index.

Python Code:

```
INF = float('inf')

def smallest_reachable_city(n, edges, threshold):
    dist = [[INF] * n for _ in range(n)]
    for i in range(n):
        dist[i][i] = 0
    for u, v, w in edges:
        dist[u][v] = w
        dist[v][u] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    best_city, best_count = -1, n + 1
    for i in range(n):
        count = sum(1 for j in range(n) if i != j and dist[i][j] <= threshold)
        if count <= best_count:
            best_count, best_city = count, i
    return best_city

# Example 1
print(smallest_reachable_city(4, [[0,1,3],[1,2,1],[2,3,4],[0,3,7]], 4))

# Example 2
print(smallest_reachable_city(5, [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]], 2))
```

Input:

Example 1:

n = 4

edges = [[0,1,3],[1,2,1],[2,3,4],[0,3,7]]

threshold = 4

Example 2:

n = 5

edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]

threshold = 2

Output:

Example 1:

3

Example 2:

0

Result:

The Python program was executed successfully and correctly identified the city with the fewest reachable cities within the given threshold for both examples.

Exercise 5: Network Delay Time

Aim:

To write a Python program that finds the minimum time required for a signal sent from a source node to reach all nodes in a network, using the Bellman-Ford algorithm, returning -1 if some node is unreachable.

Algorithm:

1. Convert the given (source, target, time) triples into a 0-indexed edge list.
2. Initialize the distance of the source vertex as 0 and the distance of all other vertices as infinity.
3. Relax all edges (u, v, w): if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v] = \text{dist}[u] + w$. Repeat this relaxation for all edges (V - 1) times, where V is the number of vertices.
4. After (V - 1) iterations, perform one additional pass over all edges to check whether any distance can still be reduced.
5. After computing the shortest distance from the source node to every other node, find the maximum value among all distances.
6. If any node's distance is still infinity, return -1 since not all nodes can be reached; otherwise, return the maximum distance as the time for the signal to reach every node.

Python Code:

```
def network_delay_time(times, n, k):
    INF = float('inf')
    edges = [(u - 1, v - 1, w) for u, v, w in times]
    dist = [INF] * n
    dist[k - 1] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    max_dist = max(dist)
    return max_dist if max_dist != INF else -1

# Example 1
print(network_delay_time([[2,1,1],[2,3,1],[3,4,1]], 4, 2))

# Example 2
print(network_delay_time([[1,2,1]], 2, 2))
```

Input:

Example 1:
times = [[2,1,1],[2,3,1],[3,4,1]]
n = 4
k = 2

Example 2:
times = [[1,2,1]]
n = 2
k = 2

Output:

Example 1:

2

Example 2:

-1

Result:

The Python program was executed successfully. In Example 1, the signal reached all nodes in 2 units of time. In Example 2, node 1 was unreachable from node 2, so the program correctly returned -1.

Exercise 6: Cheapest Flight Within K Stops

Aim:

To write a Python program that finds the minimum cost to travel from a source city to a destination city with at most K stops, using a Bellman-Ford style relaxation approach.

Algorithm:

1. Initialize the cost of the source city as 0 and all other cities as infinity.
2. Relax all the given flight edges (u, v, cost) exactly (K + 1) times, using a copy of the distance array in each round so that only up to K intermediate stops are considered.
3. In every round, for each edge (u, v, cost), if $\text{dist}[u] + \text{cost}$ is smaller than the current temporary distance to v, update the temporary distance to v.
4. After all rounds are completed, if the destination's distance is still infinity, return -1 as it is unreachable within K stops.
5. Otherwise, return the destination's distance as the minimum cost.

Python Code:

```
def find_cheapest_price(n, flights, src, dst, K):
    INF = float('inf')
    dist = [INF] * n
    dist[src] = 0
    for _ in range(K + 1):
        temp = dist[:]
        for u, v, cost in flights:
            if dist[u] != INF and dist[u] + cost < temp[v]:
                temp[v] = dist[u] + cost
        dist = temp
    return dist[dst] if dist[dst] != INF else -1
```

```
flights = [[0,1,100],[1,2,100],[2,3,100],[0,3,500]]
```

```
# Example 1 (K = 1)
print(find_cheapest_price(4, flights, 0, 3, 1))
```

```
# Example 2 (K = 2)
print(find_cheapest_price(4, flights, 0, 3, 2))
```

Input:

```
Example 1:
n = 4
flights = [[0,1,100],[1,2,100],[2,3,100],[0,3,500]]
src = 0, dst = 3, K = 1
```

```
Example 2:
n = 4
flights = [[0,1,100],[1,2,100],[2,3,100],[0,3,500]]
src = 0, dst = 3, K = 2
```

Output:

```
Example 1:
500
```

Example 2:
300

Result:

The Python program was executed successfully. With $K = 1$ stop the direct edge of cost 500 was chosen since the 2-stop route was disallowed, while with $K = 2$ stops the cheaper route through 1 and 2 costing 300 was correctly found.

Exercise 7: Detect Negative Weight Cycle

Aim:

To write a Python program that determines whether a given weighted directed graph contains a negative weight cycle using the Bellman-Ford algorithm.

Algorithm:

1. Initialize the distance of the source vertex as 0 and the distance of all other vertices as infinity.
2. Relax all edges (u, v, w): if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v] = \text{dist}[u] + w$. Repeat this relaxation for all edges (V - 1) times, where V is the number of vertices.
3. After (V - 1) iterations, perform one additional pass over all edges to check whether any distance can still be reduced.
4. If any edge can still be relaxed in this extra pass, the graph contains a negative weight cycle.
5. Otherwise, the $\text{dist}[]$ array holds the shortest distance from the source to every other vertex.

Python Code:

```
def has_negative_cycle(vertices, edges):
    dist = [0] * vertices
    for _ in range(vertices - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            return True
    return False

# Example 1
edges1 = [(0,1,1),(1,2,-1),(2,3,-1),(3,0,-1)]
print("Negative Weight Cycle Exists" if has_negative_cycle(4, edges1) else "No Negative Weight Cycle")

# Example 2
edges2 = [(0,1,4),(0,2,5),(1,2,-2),(2,3,3)]
print("Negative Weight Cycle Exists" if has_negative_cycle(4, edges2) else "No Negative Weight Cycle")
```

Input:

Example 1:
Vertices = 4
Edges = [(0,1,1),(1,2,-1),(2,3,-1),(3,0,-1)]

Example 2:
Vertices = 4
Edges = [(0,1,4),(0,2,5),(1,2,-2),(2,3,3)]

Output:

Example 1:
Negative Weight Cycle Exists

Example 2:
No Negative Weight Cycle

Result:

The Python program was executed successfully. In Example 1, the cycle 0-1-2-3-0 with total weight -2 was correctly detected as a negative weight cycle. In Example 2, no such cycle was found.

Exercise 8: Determine Reachability Between Two Vertices

Aim:

To write a Python program that determines whether a path exists between two given vertices u and v in a directed graph using Warshall's algorithm.

Algorithm:

1. Initialize the reachability matrix $reach[][]$ as a copy of the given adjacency matrix.
2. For every vertex k from 0 to $V - 1$, treat k as a possible intermediate vertex.
3. For every pair of vertices (i, j) , update $reach[i][j] = reach[i][j] \text{ OR } (reach[i][k] \text{ AND } reach[k][j])$.
4. Repeat step 3 for every pair (i, j) after fixing each intermediate vertex k .
5. The resulting matrix $reach[][]$ indicates whether a path exists between every pair of vertices (1 if reachable, 0 otherwise).

Python Code:

```
def warshall(graph):
    n = len(graph)
    reach = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    return reach

def path_exists(graph, u, v):
    reach = warshall(graph)
    return "Path Exists" if reach[u][v] else "Path Does Not Exist"
```

```
# Example 1
graph1 = [[0,1,0],[0,0,1],[0,0,0]]
print(path_exists(graph1, 0, 2))
```

```
# Example 2
graph2 = [[0,1,0],[0,0,0],[1,0,0]]
print(path_exists(graph2, 1, 2))
```

Input:

Example 1:
Graph = [[0,1,0],[0,0,1],[0,0,0]]
 $u = 0, v = 2$

Example 2:
Graph = [[0,1,0],[0,0,0],[1,0,0]]
 $u = 1, v = 2$

Output:

Example 1:
Path Exists

Example 2:
Path Does Not Exist

Result:

The Python program was executed successfully. In Example 1, a path from vertex 0 to vertex 2 was correctly found through vertex 1. In Example 2, no path from vertex 1 to vertex 2 exists, which the program correctly reported.

Exercise 9: Count Reachable Vertices from a Source

Aim:

To write a Python program that finds the number of vertices reachable from a given source vertex using the transitive closure computed by Warshall's algorithm.

Algorithm:

1. Initialize the reachability matrix `reach[][]` as a copy of the given adjacency matrix.
2. For every vertex `k` from 0 to `V - 1`, treat `k` as a possible intermediate vertex.
3. For every pair of vertices `(i, j)`, update `reach[i][j] = reach[i][j] OR (reach[i][k] AND reach[k][j])`.
4. Repeat step 3 for every pair `(i, j)` after fixing each intermediate vertex `k`.
5. The resulting matrix `reach[][]` indicates whether a path exists between every pair of vertices (1 if reachable, 0 otherwise).
6. Count the number of vertices `j` (excluding the source itself) for which `reach[source][j]` is 1, and return this count.

Python Code:

```
def warshall(graph):
    n = len(graph)
    reach = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    return reach

def count_reachable(graph, source):
    reach = warshall(graph)
    n = len(graph)
    return sum(reach[source][j] for j in range(n) if j != source)
```

```
# Example 1
graph1 = [[0,1,0,0],[0,0,1,0],[0,0,0,1],[0,0,0,0]]
print(count_reachable(graph1, 0))
```

```
# Example 2
graph2 = [[0,1,1],[0,0,1],[1,0,0]]
print(count_reachable(graph2, 1))
```

Input:

Example 1:
Graph = [[0,1,0,0],[0,0,1,0],[0,0,0,1],[0,0,0,0]]
Source = 0

Example 2:
Graph = [[0,1,1],[0,0,1],[1,0,0]]
Source = 1

Output:

Example 1:

3

Example 2:

2

Result:

The Python program was executed successfully. From vertex 0 in Example 1, vertices 1, 2 and 3 were correctly found to be reachable, giving a count of 3. From vertex 1 in Example 2, vertices 0 and 2 were correctly found to be reachable, giving a count of 2.

Exercise 10: Find the Diameter of a Graph

Aim:

To write a Python program that finds the diameter of a weighted graph, defined as the maximum shortest-path distance between any pair of vertices, using the Floyd-Warshall algorithm.

Algorithm:

1. Initialize the distance matrix $\text{dist}[][]$ as the given adjacency matrix, where $\text{dist}[i][j]$ is the direct edge weight (or infinity if no edge exists, and 0 when $i = j$).
2. For every vertex k from 0 to $V - 1$, treat k as a possible intermediate vertex.
3. For every pair of vertices (i, j) , update $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.
4. Repeat step 3 for every pair (i, j) after fixing each intermediate vertex k .
5. After all vertices have been considered as intermediates, $\text{dist}[][]$ contains the shortest distance between every pair of vertices.
6. Scan the resulting shortest-distance matrix and return the maximum finite value present, which represents the diameter of the graph.

Python Code:

```
INF = float('inf')
```

```
def floyd_warshall(graph):
    n = len(graph)
    dist = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

```
def graph_diameter(graph):
    dist = floyd_warshall(graph)
    return max(max(row) for row in dist)
```

```
# Example 1
graph1 = [[0,2,INF],[2,0,3],[INF,3,0]]
print(graph_diameter(graph1))
```

```
# Example 2
graph2 = [[0,1,4],[1,0,2],[4,2,0]]
print(graph_diameter(graph2))
```

Input:

Example 1:

```
Graph = [[0,2,INF],[2,0,3],[INF,3,0]]
```

Example 2:

```
Graph = [[0,1,4],[1,0,2],[4,2,0]]
```

Output:

Example 1:
5

Example 2:
3

Result:

The Python program was executed successfully. In Example 1, the shortest path from vertex 0 to vertex 2 through vertex 1 gave a diameter of 5. In Example 2, the diameter of the graph was correctly found to be 3.

Exercise 11: Find the Shortest Distance Between Two Vertices

Aim:

To write a Python program that finds the shortest distance between two given vertices u and v in a weighted graph using the Floyd-Warshall algorithm.

Algorithm:

1. Initialize the distance matrix $\text{dist}[][]$ as the given adjacency matrix, where $\text{dist}[i][j]$ is the direct edge weight (or infinity if no edge exists, and 0 when $i = j$).
2. For every vertex k from 0 to $V - 1$, treat k as a possible intermediate vertex.
3. For every pair of vertices (i, j) , update $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.
4. Repeat step 3 for every pair (i, j) after fixing each intermediate vertex k .
5. After all vertices have been considered as intermediates, $\text{dist}[][]$ contains the shortest distance between every pair of vertices.

Python Code:

```
INF = float('inf')

def floyd_warshall(graph):
    n = len(graph)
    dist = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist

# Example 1
graph1 = [[0,3,INF,7],[8,0,2,INF],[5,INF,0,1],[2,INF,INF,0]]
dist1 = floyd_warshall(graph1)
print(dist1[0][3])

# Example 2
graph2 = [[0,4,INF],[INF,0,6],[2,INF,0]]
dist2 = floyd_warshall(graph2)
print(dist2[2][1])
```

Input:

Example 1:
Graph = [[0,3,INF,7],[8,0,2,INF],[5,INF,0,1],[2,INF,INF,0]]
 $u = 0, v = 3$

Example 2:
Graph = [[0,4,INF],[INF,0,6],[2,INF,0]]
 $u = 2, v = 1$

Output:

Example 1:
6

Example 2:
6

Result:

The Python program was executed successfully. In Example 1, the shortest path 0-1-2-3 gave a distance of 6. In Example 2, the shortest path 2-0-1 gave a distance of 6.

Exercise 12: Find a Vertex Reachable from Every Other Vertex

Aim:

To write a Python program that finds a vertex that is reachable from every other vertex in a directed graph using Warshall's algorithm, returning the smallest indexed vertex in case of a tie.

Algorithm:

1. Initialize the reachability matrix `reach[][]` as a copy of the given adjacency matrix.
2. For every vertex `k` from 0 to `V - 1`, treat `k` as a possible intermediate vertex.
3. For every pair of vertices `(i, j)`, update `reach[i][j] = reach[i][j] OR (reach[i][k] AND reach[k][j])`.
4. Repeat step 3 for every pair `(i, j)` after fixing each intermediate vertex `k`.
5. The resulting matrix `reach[][]` indicates whether a path exists between every pair of vertices (1 if reachable, 0 otherwise).
6. For each vertex `v`, check whether `reach[u][v]` is 1 for every other vertex `u`.
7. Return the smallest indexed vertex that satisfies this condition.

Python Code:

```
def warshall(graph):
    n = len(graph)
    reach = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    return reach

def vertex_reachable_from_all(graph):
    n = len(graph)
    reach = warshall(graph)
    for v in range(n):
        if all(reach[u][v] for u in range(n) if u != v):
            return v
    return -1
```

```
# Example 1
graph1 = [[0,1,1],[0,0,1],[0,0,0]]
print(vertex_reachable_from_all(graph1))
```

```
# Example 2
graph2 = [[0,1,0],[0,0,1],[1,0,0]]
print(vertex_reachable_from_all(graph2))
```

Input:

Example 1:
Graph = [[0,1,1],[0,0,1],[0,0,0]]

Example 2:
Graph = [[0,1,0],[0,0,1],[1,0,0]]

Output:

Example 1:
2

Example 2:
0

Result:

The Python program was executed successfully. In Example 1, vertex 2 was correctly found to be reachable from both vertices 0 and 1. In Example 2, since every vertex is reachable from every other vertex, the smallest indexed vertex 0 was correctly returned.

Exercise 13: Find the Vertex with Maximum Reachability

Aim:

To write a Python program that finds the vertex that can reach the maximum number of other vertices in a directed graph using Warshall's algorithm, returning the smallest indexed vertex in case of a tie.

Algorithm:

1. Initialize the reachability matrix `reach[][]` as a copy of the given adjacency matrix.
2. For every vertex `k` from 0 to `V - 1`, treat `k` as a possible intermediate vertex.
3. For every pair of vertices `(i, j)`, update `reach[i][j] = reach[i][j] OR (reach[i][k] AND reach[k][j])`.
4. Repeat step 3 for every pair `(i, j)` after fixing each intermediate vertex `k`.
5. The resulting matrix `reach[][]` indicates whether a path exists between every pair of vertices (1 if reachable, 0 otherwise).
6. For each vertex `v`, count the number of vertices reachable from it using the `reach[][]` matrix.
7. Return the vertex with the maximum count; if multiple vertices tie, return the smallest indexed one.

Python Code:

```
def warshall(graph):
    n = len(graph)
    reach = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    return reach
```

```
def vertex_max_reachability(graph):
    n = len(graph)
    reach = warshall(graph)
    best_v, best_c = 0, -1
    for v in range(n):
        c = sum(reach[v][j] for j in range(n) if j != v)
        if c > best_c:
            best_c, best_v = c, v
    return best_v
```

```
# Example 1
graph1 = [[0,1,1,0],[0,0,1,0],[0,0,0,1],[0,0,0,0]]
print(vertex_max_reachability(graph1))
```

```
# Example 2
graph2 = [[0,1,0],[0,0,1],[1,0,0]]
print(vertex_max_reachability(graph2))
```

Input:

Example 1:
Graph = [[0,1,1,0],[0,0,1,0],[0,0,0,1],[0,0,0,0]]

Example 2:
Graph = [[0,1,0],[0,0,1],[1,0,0]]

Output:

Example 1:

0

Example 2:

0

Result:

The Python program was executed successfully. In Example 1, vertex 0 could reach vertices 1, 2 and 3, giving it the maximum reachability. In Example 2, all vertices could reach two other vertices, so the smallest indexed vertex 0 was correctly returned.

Exercise 14: Find the Diameter of a Weighted Graph

Aim:

To write a Python program that finds the diameter of a weighted graph, defined as the maximum shortest-path distance between any pair of vertices, using the Floyd-Warshall algorithm.

Algorithm:

1. Initialize the distance matrix $\text{dist}[][]$ as the given adjacency matrix, where $\text{dist}[i][j]$ is the direct edge weight (or infinity if no edge exists, and 0 when $i = j$).
2. For every vertex k from 0 to $V - 1$, treat k as a possible intermediate vertex.
3. For every pair of vertices (i, j) , update $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.
4. Repeat step 3 for every pair (i, j) after fixing each intermediate vertex k .
5. After all vertices have been considered as intermediates, $\text{dist}[][]$ contains the shortest distance between every pair of vertices.
6. Scan the resulting shortest-distance matrix and return the maximum finite value present, which represents the diameter of the graph.

Python Code:

```
INF = float('inf')
```

```
def floyd_warshall(graph):
    n = len(graph)
    dist = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

```
def graph_diameter(graph):
    dist = floyd_warshall(graph)
    return max(max(row) for row in dist)
```

```
# Example 1
graph1 = [[0,2,INF],[2,0,4],[INF,4,0]]
print(graph_diameter(graph1))
```

```
# Example 2
graph2 = [[0,1,5],[1,0,2],[5,2,0]]
print(graph_diameter(graph2))
```

Input:

Example 1:

```
Graph = [[0,2,INF],[2,0,4],[INF,4,0]]
```

Example 2:

```
Graph = [[0,1,5],[1,0,2],[5,2,0]]
```

Output:

Example 1:
6

Example 2:
3

Result:

The Python program was executed successfully. In Example 1, the shortest path from vertex 0 to vertex 2 through vertex 1 gave a diameter of 6. In Example 2, the diameter of the graph was correctly found to be 3.

Exercise 15: Check Whether a Graph is Strongly Connected

Aim:

To write a Python program that determines whether a directed graph is strongly connected, that is, whether every vertex is reachable from every other vertex, using Warshall's algorithm.

Algorithm:

1. Initialize the reachability matrix `reach[][]` as a copy of the given adjacency matrix.
2. For every vertex `k` from 0 to `V - 1`, treat `k` as a possible intermediate vertex.
3. For every pair of vertices `(i, j)`, update `reach[i][j] = reach[i][j] OR (reach[i][k] AND reach[k][j])`.
4. Repeat step 3 for every pair `(i, j)` after fixing each intermediate vertex `k`.
5. The resulting matrix `reach[][]` indicates whether a path exists between every pair of vertices (1 if reachable, 0 otherwise).
6. Check every pair `(i, j)` with `i` not equal to `j`; if `reach[i][j]` is 0 for any such pair, the graph is not strongly connected.
7. If `reach[i][j]` is 1 for every pair, the graph is strongly connected.

Python Code:

```
def warshall(graph):
    n = len(graph)
    reach = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    return reach

def is_strongly_connected(graph):
    n = len(graph)
    reach = warshall(graph)
    for i in range(n):
        for j in range(n):
            if i != j and not reach[i][j]:
                return False
    return True

# Example 1
graph1 = [[0,1,0],[0,0,1],[1,0,0]]
print("Strongly Connected" if is_strongly_connected(graph1) else "Not Strongly Connected")

# Example 2
graph2 = [[0,1,0],[0,0,1],[0,0,0]]
print("Strongly Connected" if is_strongly_connected(graph2) else "Not Strongly Connected")
```

Input:

Example 1:

Graph = [[0,1,0],[0,0,1],[1,0,0]]

Example 2:

Graph = [[0,1,0],[0,0,1],[0,0,0]]

Output:

Example 1:
Strongly Connected

Example 2:
Not Strongly Connected

Result:

The Python program was executed successfully. In Example 1, every vertex could reach every other vertex, so the graph was correctly identified as strongly connected. In Example 2, vertex 2 could not reach vertices 0 or 1, so the graph was correctly identified as not strongly connected.